

SISTEMI OPERATIVI LAB.

Il **SISTEMA OPERATIVO** è un software che funge da interfaccia tra utente e hardware; più in particolare gestisce le risorse hardware e software.

Un SO ha diversi **componenti principali**:

- **Driver** dei device (scritti in c ed assembly), che servono per far funzionare l'hardware
- **Kernel** (nucleo), che tratta la schedulazione della CPU, la gestione della memoria ecc.
- **Shell** (guscio), che permette l'interazione con il SO (da parte dell'utente o di programmi)
- **File system**, che descrive come sono organizzati i file all'interno della macchina
- **Applicazioni** usate dall'utente

Più nello specifico:

- Il **KERNEL** si interfaccia direttamente all'hardware e offre delle operazioni (**system-call**) agli utenti finali per poter sfruttare l'hardware stesso.
- La **SHELL** permette l'interazione con il kernel; più in particolare interpreta i comandi che si scrivono a riga di comando: a seconda del comando, permette all'utente di accedere ai vari servizi che il kernel offre.

`cat /etc/shells` elenca le shells presenti all'interno del nostro SO

`echo $SHELL` visualizza la shell attualmente in esecuzione

Le **SYSTEM CALLS** ("syscalls" abbreviato), sono dei punti di accesso ai servizi che il kernel offre (lettura file, scrittura file, ...) e sono identificate da numeri.

Con il comando `strace <command>` è possibile avere l'elenco di tutte le system calls che vengono utilizzate per effettuare una operazione (`<command>`). Ad esempio `strace echo ciao`.

Con `strace -wC <command>` si ha tale elenco sintetizzato in una tabella.

Con il comando `man <command>` è possibile invocare e mostrare sul terminale la *manual page* collegata a quello specifico comando. Es: `man printf`

Le **MAN PAGE** sono divise in varie sezioni (descrizione del comando, syscalls, librerie...); aggiungendo il numero di tale sezione dopo il comando, è possibile accedere direttamente alla sezione scelta.

Es: `man 1 printf`

1	General commands
2	System calls
3	Library functions, covering in particular the C standard library
4	Special files (usually devices, those found in /dev) and drivers
5	File formats and conventions
6	Games and screensavers
7	Miscellanea
8	System administration commands and daemons

I **FILE** sono una sequenza di Byte e sono organizzati in strutture dati ad albero.

Il **FYLE SYSTEM** è tutto il sistema di gestione dei file, che il SO possiede:

- C'è sempre una **directory principale (root)** con all'interno file o altre sotto-directory.
- La **directory radice** dell'albero è "/"
- La **directory corrente** è indicata con "."
- La **directory padre** della cartella corrente è indicata con ".."
- La **directory home** dell'utente è "~"
- Le directory si separano con con uno "/"
- Il "**full-pathname**" è tutto il percorso di un file (dalla directory principale fino al file stesso).

Più in generale, ci sono **7 Tipi di File**:

- (marked by "-" in ls -l) **regular file**: contains data, are on disk
- (marked by "d" in ls -l) **directories**: contains names of other files
- (marked by "c" in ls -l) **character special file**: used to read/write devices byte by byte
- (marked by "b" in ls -l) **block special file**: used to read/write to devices in block (disks).
- (marked by "p" in ls -l) **FIFO**: a special file used for interprocess communication (IPC)
- (marked by "s" in ls -l) **socket**: used for network communication
- (marked by "l" in ls -l) **symbolic link**: it just points to another file

Con il comando `stat <file>` è possibile vedere i dettagli di un qualsiasi tipo di file (device, ultimo accesso, ecc...)

Con il comando `ls -l <file>` è possibile vedere il contenuto di una directory.

```
marco@marco-VirtualBox:~/Desktop$ ls -l
total 60
-rwxrwxr-x 1 marco marco 15960 Sep 28 17:18 a.out
-rwxrwxr-x 1 marco marco 15960 Sep 28 17:19 hello
-rw-rw-r-- 1 marco marco 74 Sep 28 17:00 hello.c
-rwxrwxr-x 1 marco marco 15960 Sep 28 18:22 var
-rw-rw-r-- 1 marco marco 167 Sep 28 18:04 var.c
-rw-rw-r-- 1 marco marco 1704 Sep 28 18:04 var.o
```

Analizzando la prima lettera è possibile capire il tipo di file; in questo caso, sono tutti file

Le directory principali ed il loro contenuto sono:

/bin	common programs, executables (often subdirectory of /usr or /usr/local)
/boot	The startup files and the kernel
/etc	contains configuration files
/home	parent of home directory of common users
/tmp	place for temporary files, writable by everybody, cleaned upon reboot
/root	home directory of the administrator
/lib	library files
/proc	information on processes and resources (only on some Unix-like machines)
/dev	contains references to special files (disks, terminals, etc.)

Il SO Linux è un **SISTEMA MULTI-ACCOUNT**.

Ci sono 3 diverse **Tipologie di Account** (con altrettante diverse liste di permessi):

- **Account root (ID:0)** -> può fare tutto
- **Account utente**
- **Account utente di sistema (ID <1000)**

Per vedere l'elenco degli utenti presenti, si utilizza il comando `cat /etc/passwd`

In Linux esiste anche il concetto di "**gruppo**" che ci viene in aiuto se ad esempio si vogliono assegnare permessi speciali solo ad alcuni utenti. Con il comando `cat /etc/group` si può visualizzare l'elenco dei gruppi presenti.

Facendo `sudo <command>`, posso eseguire tale comando come amministratore (account root); ciò è possibile solo se tale utente appartiene al gruppo "sudo".

Per passare all'account root posso utilizzare il comando "`sudo su`": mi verrà chiesta la password dell'account root

Per ogni tipo di file ci sono **3 Permessi**:

- **Permesso di Lettura**
- **Permesso di Scrittura**
- **Permesso di Esecuzione**

Posso assegnare tali permessi agli utenti, a un gruppo oppure a tutti.

```
marco@marco-VirtualBox:~$ ls -l
total 36
drwxr-xr-x 2 marco marco 4096 Sep 28 18:22 Desktop
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Documents
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Downloads
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Music
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Pictures
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Public
drwx----- 3 marco marco 4096 Sep 27 21:35 snap
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Templates
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Videos
```

Con il comando `ls -l <file>` posso sapere quali permessi ho su uno specifico file, per ogni utente/gruppo/utente del sistema.

Analizzando le lettere successive dopo la prima, posso vedere i permessi che ho su quel file ("r"-> read, "w"->write, "x"-> exe)

Per modificare i permessi utilizzo il comando:

`chmod <tipoUtente> <tipoOperazione> <permessi> <file>`, dove:

`<tipoUtente>` può essere "u"->user, "g"->group, "o"->all

`<tipoOperazione>` può essere "+"-> aggiungere permesso, "-"->rimuovere permesso

`<permessi>` può essere "r"->read, "w"->write, "x"->exe

Variante di questo comando è utilizzare le cifre in ottale (da 0 a 7) per indicare ogni singola combinazione per ogni diversa categoria di utente:

user			group			other			octal
r	w	x	r	w	x	r	w	x	
1	1	1	1	1	0	1	0	0	=764

Equivalent commands

► `chmod u=rwx,g=rw,o=r <filename>`

► `chmod 764 <filename>`

È inoltre possibile:

- **Ridirigere l'Input** di un comando attraverso l'operatore **>** (*maggiore*)
- **Ridirige l'Output** di un comando attraverso l'operatore **<** (*minore*)
- **Ridirigere l'Output di un Comando A come input di un Comando B**, con l'operatore **|** (*pipe*)

```
marco@marco-VirtualBox:~$ ls -l
total 36
drwxr-xr-x 2 marco marco 4096 Sep 28 18:22 Desktop
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Documents
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Downloads
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Music
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Pictures
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Public
drwx----- 3 marco marco 4096 Sep 27 21:35 snap
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Templates
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Videos
marco@marco-VirtualBox:~$ ls -l > file.txt
marco@marco-VirtualBox:~$ cat file.txt
total 36
drwxr-xr-x 2 marco marco 4096 Sep 28 18:22 Desktop
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Documents
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Downloads
-rw-rw-r-- 1 marco marco 0 Sep 28 18:55 file.txt
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Music
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Pictures
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Public
drwx----- 3 marco marco 4096 Sep 27 21:35 snap
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Templates
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Videos
```

Esempio (Ridirigere l'Input):

`ls -l` mostra a video l'elenco;
`ls -l > file.txt` salvo l'elenco in "file.txt"

Esempio (Ridirigere l'Output):

con `wc < file.txt` mostro a video il numero di righe, parole e byte di "file.txt"

```
marco@marco-VirtualBox:~$ wc < file.txt
11  92 520
```

Esempio (Ridirigere l'Output di un Comando A come input di un Comando B):

con `grep` faccio il match di stringhe

```
marco@marco-VirtualBox:~$ ls -l | grep Music
drwxr-xr-x 2 marco marco 4096 Sep 27 21:35 Music
```

Comandi comuni per navigare nel filesystem:

cat	Concatenate: displays a file.
cd	Change directory: moves you to the directory identified
cp	Copy: copies one file/directory to specified location.
file	Identifies the file type (binary, text, etc).
find	Finds a file/directory.
head	Shows the beginning of a file.
less	Browses through a file from end or beginning.
ls	List: shows the contents of the directory specified.
mkdir	Make directory: creates the specified directory.
more	Browses through a file from beginning to end.
mv	Move: moves the location of or renames a file/directory.
pwd	Print working directory: shows the current directory the user is in.
rm	Remove: removes a file.
tail	Shows the end of a file.
touch	Creates a blank file or modifies an existing file's attributes.

Introduzione al Corso

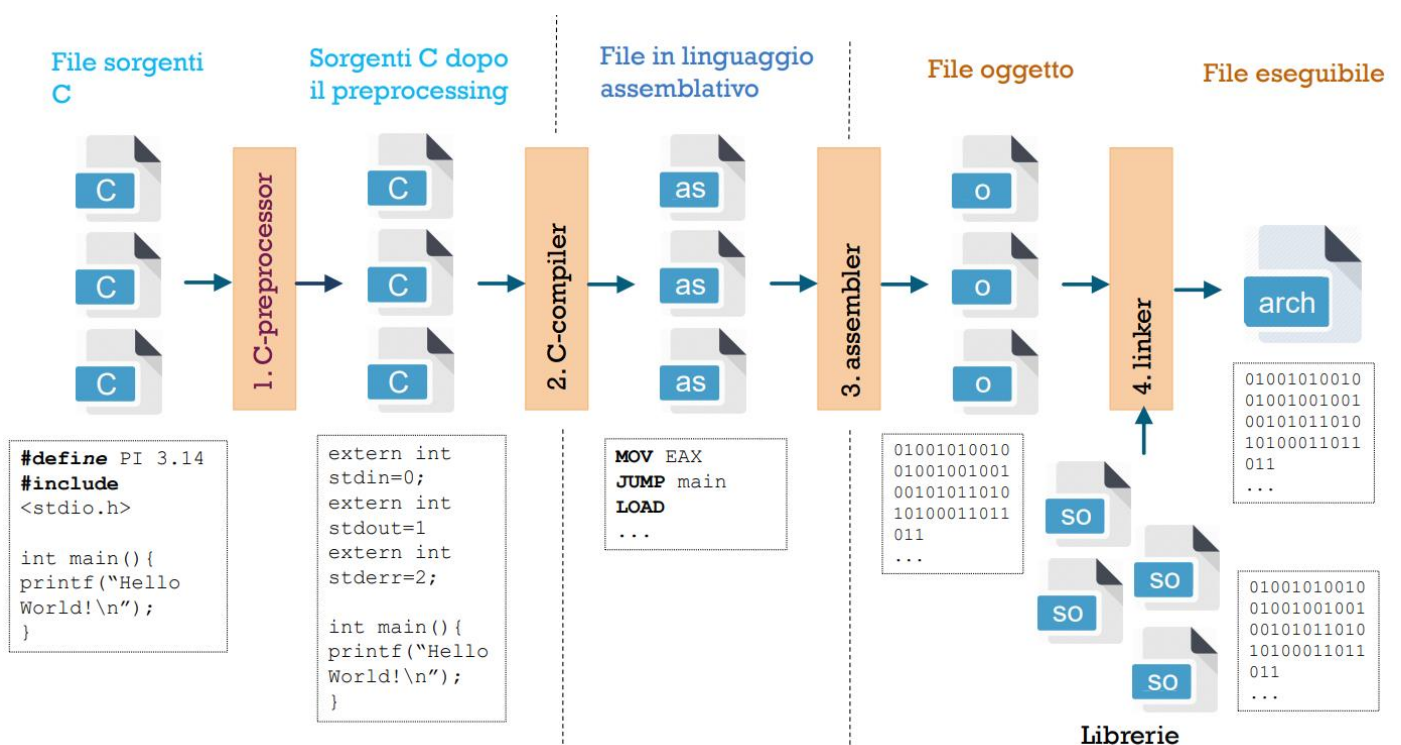
Il corso introdurrà alla programmazione in linguaggio C in **ambiente UNIX**.
Considerando i tre sistemi operativi più diffusi in ambito educativo:

- *Linux* e *OSX Apple* sono sistemi operativi UNIX-like
- *Microsoft Windows* non è UNIX-like.
 - Esistono però delle librerie di adattamento a Linux (...) pur rimanendo in ambiente Windows; scelta non consigliata!



Diremo che la **macchina C** interpreta ed esegue programmi scritti in **linguaggio C**;
In realtà l'esecuzione dei programmi C è ottenuta attraverso un processo di compilazione.

- Il **C** è un linguaggio non comprensibile all'architettura hardware del processore: questa è in grado di eseguire solo programmi informatici scritti in **linguaggio macchina**;
- Un programma in C dovrà essere quindi tradotto in un programma equivalente in linguaggio macchina specifico di una **architettura target di processore** (esempi: Intel x86, ARM, etc.), solo quest'ultimo potrà essere eseguito.
- Il processo di traduzione si chiama **compilazione**.
- In realtà il processo di compilazione si compone di diversi passaggi:
 1. **Pre-processing** dei file C (**preprocessore-C**);
 2. Traduzione o **compilazione** dei file sorgenti C pre-processati in sorgenti in linguaggio assembly (**compilatore-C**);
 3. **Traduzione** dal linguaggio assembly al linguaggio macchina (**assembler**), i file prodotti contengono **codice oggetto**;
 4. **Linking** dei file (**linker**).



GCC - Gnu Compiler Collection

gcc non è un compilatore C, è uno strumento front-end usato in tutto il processo di produzione di un programma eseguibile a partire da sorgenti scritti in linguaggi di programmazione di alto livello, secondo lo schema illustrato precedentemente;

- Noi lo useremo per produrre un eseguibile di un progetto scritto in C;
- **gcc** si utilizza da linea di comando
- In generale **gcc** prende in input dei file e produce in output altri file.
- Attraverso le **opzioni** il programmatore specifica cosa deve fare (-o, -E, -c, -S ...)
- Inoltre, oltre alle opzioni **gcc** usa le estensioni dei nomi dei file forniti sulla linea di comando per capire come operare.

È possibile verificare la presenza del gcc sul nostro computer, utilizzando il comando `gcc -v`. Se non è installato, è possibile farlo tramite `sudo apt install gcc`.

In generale noi compileremo un programma con `gcc <filename>`; se non ci sono errori, il programma non viene immediatamente eseguito, bensì viene creato di default un `a.out` e per eseguirlo è necessario eseguire un ulteriore comando: `./a.out`.

È spesso però preferito nominare tale file con un nome simile a quello del file sorgente.

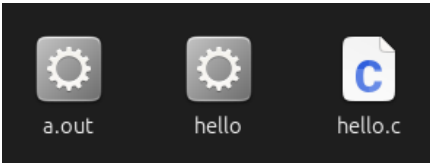
È possibile scegliere il nome che vogliamo dare al nostro file output aggiungendo l'opzione `-o`

Esempio:

```
marco@marco-VirtualBox:~/Desktop$ gcc hello.c
marco@marco-VirtualBox:~/Desktop$ ./a.out
Hello World!
marco@marco-VirtualBox:~/Desktop$ gcc hello.c -o hello
marco@marco-VirtualBox:~/Desktop$ ./hello
Hello World!
```

```
#include <stdio.h>

int main(){
    printf("Hello World!\n");
    return 0;
}
```



- I flag `-E` `-c` `-S` servono a fermare il processo di traduzione in una delle fasi intermedie:
 - `-E` ferma la traduzione subito dopo il pre-processing.
 - `-c` ferma la traduzione subito dopo l'assemblaggio (compila solo);
 - `-S` ferma la traduzione subito dopo la compilazione (non assembla);

Ricapitolando:

- Il C nasce come linguaggio per lo sviluppo del sistema operativo UNIX
- Linguaggio compilato, ovvero, il codice sorgente viene tradotto interamente da un compilatore in codice eseguibile. L'esecuzione può avvenire solo dopo che la compilazione è terminata senza errori.

Un programma in sorgente C è composto generalmente da due tipi diversi di file:

- **file di intestazione (header file)**: questi non contengono istruzioni C ma **dichiarazioni di dati** e **definizioni di tipo**. Il nome di questi file deve avere estensione **h**. Es: calcolatrice.h
- **file sorgente C**: questi contengono principalmente istruzioni C organizzate in funzioni nonché **definizioni di dati**. Il nome di questi file deve avere estensione **c**. Es: calcolatrice.c

Il 1° esempio di programma in C è il seguente:

```
#include <stdio.h>

int main(){
    printf("Hello World!\n");
}
```

Il carattere **#** introduce una **direttiva** per il **pre-processore**. Nel codice dell'esempio si tratta della direttiva `include`, ossia di *inclusione di file*; in questo caso del file `stdio.h`. Tecnicamente il **pre-processore C** sostituisce alla direttiva il contenuto del file **stdio.h**.

- Esistono anche altre direttive: noi vedremo la **include**, **define** e **ifdef**.
-
- Il punto di ingresso, detto **entry-point**, di un programma C è la funzione `main()`.
 - Ogni programma C deve avere esattamente una sola definizione di funzione `main()`.
 - Le funzioni C a conclusione della loro esecuzione restituiscono un **valore**; ogni valore in C è di un determinato **tipo primitivo**; il C è un linguaggio tipato. In questo caso, la funzione `main()` restituisce un valore di tipo **int**.
 - Tra parentesi graffe segue la **definizione** della funzione; la porzione di codice tra le parentesi graffe è anche detto *testo* della funzione.

Più in generale la struttura tipica di un programma in C è la seguente:

inclusione di file e direttive al preprocessore
definizione di tipi
dichiarazioni di variabili globali
definizione di funzioni
funzione <code>main()</code>

Variabili

Una **VARIABILE** in C è un'area di memoria. Potremmo dire che è un modo comodo per vedere la memoria.

In C la **MEMORIA** è una sequenza astratta di Byte.

Gli **indirizzi di memoria** sono rappresentati da un certo numero di Byte. Es: 8 Byte ovvero 16 cifre esadecimali; segue che lo **spazio di indirizzamento** è $2^{8 \times 8} = 2^{64}$

address	content
...	...
7FFF0040671A8107	A7
7FFF0040671A8108	E8
7FFF0040671A8109	03
7FFF0040671A810A	00
7FFF0040671A810B	00
7FFF0040671A810C	50
...	...

Per memorizzare una variabile più grande in memoria bisogna memorizzare i dati in maniera contigua; per farlo abbiamo due scelte:

- **little-endian**: dal Byte meno significativo al più significativo
- **big-endian**: dal Byte più significativo al meno significativo

...	...	
7FFF0040671A8108	E8	var
7FFF0040671A8109	03	
7FFF0040671A810A	00	
7FFF0040671A810B	00	

...	...	
7FFF0040671A8108	00	var
7FFF0040671A8109	00	
7FFF0040671A810A	03	
7FFF0040671A810B	E8	

In molte architetture si utilizza la modalità *little-endian*

Il tipo **tipi primitivi** in C sono i seguenti:

- **int, short int, long int, unsigned int, unsigned short int, unsigned long int**
- **float, double**
- **char** -> Attenzione! È un intero in realtà: è possibile farci operazioni aritmetiche

Infine:

- Il tipo **void** è usato per dichiarare una funzione che non restituisce alcun valore alla terminazione.
- Esiste il tipo **puntatore** a **indirizzo di memoria**: *****, che si accompagna al nome di un tipo primitivo. Ad esempio: **int ***, **void ***, **double ***, ...
 - Il puntatore a valori di tipo void (**void ***) indica un puntatore ad un dato generico.
 - Si possono anche definire variabili che sono puntatori a puntatori. Es: **int ****, **void ****

N.B non esiste il valore booleano!

Il C, con eccezione per il tipo **char**, non stabilisce alcuna dimensione dei dati in base al loro tipo; la dimensione è *machine-dependent*, cioè dipendente dall'architettura per la quale è generato il testo eseguibile del programma. A tale scopo può essere utile l'operatore **sizeof()**

```
#include <stdio.h>

int main(){
    int a;

    printf("INTEGER TYPES IN C\n");
    printf(" Bytes to represent a char: %d\n", (int)sizeof(char));
    printf(" Bytes to represent a short: %d\n", (int)sizeof(short));
    printf(" Bytes to represent a int: %d\n", (int)sizeof(a));
    printf(" Bytes to represent a long: %d\n", (int)sizeof(long));

    printf("\nFLOATING POINT TYPES IN C\n");
    printf(" Bytes to represent a float: %d\n", (int)sizeof(float));
    printf(" Bytes to represent a double: %d\n", (int)sizeof(double));

    return 0;
}
```

Attenzione: **sizeof()** non è una funzione; bensì è un operatore!

Il valore di una variabile è il complemento a 2 (con segno) dei suoi Byte


```
int i;           // definizione senza inizializzazione
short j = 2;     // definizione con inizializzazione
int *ptr = NULL; // definizione di un puntatore con inizializzazione
int k = i+j%2;   // k == ?
```

NOTE:

- Una variabile deve essere dichiarata prima di poter essere utilizzata
- Dichiarare una variabile non genera alcun codice; serve al compilatore per sapere quanta memoria riservare ad essa.
- **NULL** è una costante simbolica; indica il valore di puntatore non valido. Non fa parte del C.
 - Le costanti simboliche sono definite dal programmatore attraverso l'utilizzo del pre-processore C.

La **definizione** del C in generale è: **tipo** nome [= expr];

- expr può essere una qualsiasi espressione C.
- Una espressione ha un valore ed ogni valore è di un determinato tipo primitivo.
- Nella inizializzazione di una variabile il tipo di expr deve accordarsi con il tipo della variabile: **type matching**.
 - Nell'assegnamento di un valore ad una variabile, il compilatore C ammette (quando non c'è perdita di precisione) alcune **conversioni implicite di tipo** al fine di garantire il matching.
 - Se c'è perdita di precisione la conversione è consentita, potremmo però avere degli avvertimenti (*warnings*) durante la compilazione.
 - Se i tipi sono incompatibili il compilatore termina con un errore di **type mismatch**.

```
#include <stdio.h>

int main(){
    int i=9;

    double a=2.0*i/2;
    double b=2.0*(i/2);
    double c=2.0*(i/2.0);
    printf("a:%lf, b:%lf, c:%lf\n",a,b,c);
    return 0;
}
```

NOTE: le espressioni sono valutate da sinistra a destra rispettando la priorità e l'associatività degli operatori e l'ordine imposto dalle parentesi.

a:9.000000, b:8.000000, c:9.000000

- In C una variabile non inizializzata ha un valore **indefinito**:
 - Mentre in Java il compilatore genera un errore quando riscontra l'uso di una variabile indefinita, in C questo non avviene; il compilatore potrebbe al più generare un **warning**.
 - Questo valore indefinito dipende dal runtime.

- Una variabile può essere usata solo dopo la sua **dichiarazione**.
- Una porzione di codice racchiusa tra una coppia di graffe bilanciate è chiamata *blocco*.

```
{ // Blocco A
  int i;
  int k=3;
  ....
  { // Blocco A.1
    int i;
    int j;
    i+=1; // è la i definita in A.1
    ....
    k+=1;
  }
  i+=2; // è la i definita in A
  j=3;  // errore, variabile non definita
}

{ // Blocco B
  int i;
  i++;
}
```

- In ogni blocco il codice andrebbe rientrato con un carattere di tabulazione aggiuntivo;
- I blocchi possono essere annidati gli uni dentro gli altri;
- Le dichiarazioni delle variabili Idealmente andrebbero inserite al principio di ogni blocco;
- La variabile i definita in A è diversa dalla variabile i definita in B, inoltre è visibile solo dalla sua definizione fino al termine del blocco;
- La variabile i in A.1 **nasconde** la variabile i in A.

I/O - printf e scanf

In C esistono alcune funzioni della libreria standard adibite all'input e all'output formattato, rispettivamente sono:

```
printf(char *format, ...) // prints on standard output stdout
scanf(char *format, ...)  // reads from standard input stdin
```

Entrambe sono associate allo **standard input**, per cui è necessaria la direttiva `#include <stdio.h>`

- Entrambe prendono un numero variabile di parametri;
- Il primo parametro è detto *stringa di formato*.
 - Nel caso della printf costituisce nell'uso più semplice il testo da stampare.
 - Alcuni caratteri particolari sono indicati nella stringa con **sequenze di escape**. Queste sono:

<code>\n</code>	line feed	<code>\r</code>	carriage return	<code>\t</code>	tabulazione	<code>\f</code>	form feed
<code>\b</code>	backspace	<code>\\</code>	carattere \	<code>\"</code>	carattere "	<code>%%</code>	carattere %

- La stringa può contenere dei **caratteri di controllo** %:
 - questi non sono stampati ma indicano delle posizioni dove stampare, secondo l'ordine di lettura, i valori di espressioni fornite nei parametri rimanenti della chiamata a funzione.
 - Dopo il carattere di controllo % seguono uno o più caratteri che identificano il **formato di conversione** secondo il quale stampare il valore:

<code>%d %i</code>	per gli interi
<code>%u</code>	per gli interi unsigned
<code>%ld</code>	per gli interi long (long decimal)
<code>%c</code>	per i caratteri
<code>%s</code>	per le stringhe
<code>%f %lf</code>	per i float e i double
<code>%e %E</code>	notazione esponenziale
<code>%g %G</code>	notazione migliore tra <code>%f</code> e <code>%e</code> (rispettivamente <code>%f</code> e <code>%E</code>)
<code>%p</code>	per valori puntatore

Es:

```
int n = 234; // total number of students
printf("In aula ci sono %d studenti.\n", n);
```

- I caratteri di formato possono essere preceduti da due numeri separati da un punto. Essi indicano quante cifre usare rispettivamente per la parte intera e decimale di un numero.

Es:

```
double r = rand()/(double)RAND_MAX; // random number between [0,1]
printf("Random number is: %.3lf\n", r);
```